```
/*
 Septembar - 2023

 Koristeci WCF kreirati sistem za prijavu ispita. Potrebno je podrzati mogucnost:
    - Prijava ispita - Student(indeks, ime i prezime), lista predmeta(naziv i sifra) koju prijavljuje, rok
    - Odjava isputa - Student i predmet koji odjavljuje, sistem vraca da li je uspesno odjavio ispit ili gresku ako je do nje doslo.
    - Svi prijavljeni ispiti za studenta i ispitni rok
    - Broj studenata po ispitu u ispitnom roku
    - Sve studente koji su prijavili ispit u ispitnom roku
 */

namespace WCF.Contracts
{
    [ServiceContract]
    public interface IExamRegistrationService
    {
        [OperationContract]
        OperationResult RegisterExam(Student student, List<Subject> subjects, string examPeriod);

        [OperationContract]
        OperationResult UnregisterExam(Student student, Subject subject);

        [OperationContract]
        List<Exam> GetStudentExams(string studentIndex, string examPeriod);
        list<string>
        [OperationContract]
        int GetStudentCountForExam(string examPeriod, Subject subject);

        [OperationContract]
        List<Student> GetStudentsForExam(string examPeriod, Subject subject);
    }

    [DataContract]
    public class Student
    {
        [DataMember]
        public string Index { get; set; }

        [DataMember]
        public string FullName { get; set; }
    }

    [DataContract]
    public class Subject
    {
        [DataMember]
        public string Code { get; set; }

        [DataMember]
        public string Name { get; set; }
    }

    [DataContract]
    public class Exam
    {
        [DataMember]
        public Student Student { get; set; }

        [DataMember]
        public Subject Subject { get; set; }

        [DataMember]
        public string ExamPeriod { get; set; }
    }

    [DataContract]
    public class OperationResult
    {
        [DataMember]
        public bool IsError { get; set; }

        [DataMember]
        public string ErrorMessage { get; set; }

        [DataMember]
        public string SuccessMessage { get; set; }
    }

}

namespace WCF.Services
{
    [ServiceBehavior(InstanceContextMode = InstanceContextMode.Single)]
    public class ExamRegistrationService : IExamRegistrationService
    {
        private readonly List<Exam> registeredExams = new List<Exam>();

        public OperationResult RegisterExam(Student student, List<Subject> subjects, string examPeriod)
        {
            OperationResult operationResult = new OperationResult();

            foreach (var subject in subjects)
            {
                // Check if the student has already registered for the same subject in the exam period
                if (IsStudentAlreadyRegistered(student, subject, examPeriod))
                {
```

```
                operationResult.IsError = true;
                operationResult.ErrorMessage = $"Student {student.FullName} has already registered for {subject.Name} in {examPeriod}";
                break;
            }

            var exam = new Exam
            {
                Student = student,
                Subject = subject,
                ExamPeriod = examPeriod
            };
            registeredExams.Add(exam);
        }

        if (!operationResult.IsError)
        {
            operationResult.SuccessMessage = "Registration successful!";
        }

        return operationResult;
    }

    private bool IsStudentAlreadyRegistered(Student student, Subject subject, string examPeriod)
    {
        return registeredExams.Any(e =>
            e.Student.Index == student.Index &&
            e.Subject.Code == subject.Code &&
            e.ExamPeriod == examPeriod);
    }

    public OperationResult UnregisterExam(Student student, Subject subject)
    {
        OperationResult operationResult = new OperationResult();

        var examToRemove = registeredExams.FirstOrDefault(e =>
            e.Student.Index == student.Index && e.Subject.Code == subject.Code);

        if (examToRemove != null)
        {
            registeredExams.Remove(examToRemove);
            operationResult.SuccessMessage = $"Successfully unregistered {examToRemove.Subject.Name}";
        }
        else
        {
            operationResult.IsError = true;
            operationResult.ErrorMessage = $"No registration found for Student {student.FullName} and Subject {subject.Name}";
        }

        return operationResult;
    }

    public List<Exam> GetStudentExams(string studentIndex, string examPeriod)
    {
        return registeredExams
            .Where(e => e.Student.Index == studentIndex && e.ExamPeriod == examPeriod)
            .ToList();
    }

    public int GetStudentCountForExam(string examPeriod, Subject subject)
    {
        return registeredExams.Count(e => e.ExamPeriod == examPeriod && e.Subject.Code == subject.Code);
    }

    public List<Student> GetStudentsForExam(string examPeriod, Subject subject)
    {
        return registeredExams
            .Where(e => e.ExamPeriod == examPeriod && e.Subject.Code == subject.Code)
            .Select(e => e.Student)
            .Distinct()
            .ToList();
    }
  }
}

// Web.config
<system.serviceModel>
  <services>
    <service name="WCF.Services.ExamRegistrationService">
      <endpoint address="/Services/ExamRegistrationService.svc" binding="basicHttpBinding" contract="WCF.Contracts.IExamRegistrationService" />
    </service>
  </services>
</system.serviceModel>

namespace ClientConsoleApp
{
    internal class ExamRegistrationConsoleClient
    {
        public ExamRegistrationConsoleClient()
        {
            using (ExamRegistrationServiceClient examRegistrationServiceClient = new ExamRegistrationServiceClient())
            {
                RegisterExam(examRegistrationServiceClient, out Student student, out string examPeriod);

                GetStudentExams(examRegistrationServiceClient, student, examPeriod);

                Subject examSubject = GetStudentCountForExam(examRegistrationServiceClient, examPeriod);
```

List<string> examinfo = new List<string>();
var exams = registeredExams
.Where(e => e.ExamPeriod == examPeriod && e.Subject.Code == subject.Code)
.GroupBy(e => e.Subject.Name)
.Select(g => new {Subject = g.Key, StudentCount = g.Count()});

foreach (var exam in exams)
{ string examinfo = $"{exam.Subject}, Broj Prijavljenih {exam.StudentCount}";
examinfo.Add(examinfo);
}
return examinfo;
}

List<string>

```
            GetStudentsForExam(examRegistrationServiceClient, examPeriod, examSubject);

            UnregisterExam(examRegistrationServiceClient, student);
        }
    }

    private static void RegisterExam(ExamRegistrationServiceClient examRegistrationServiceClient, out Student student, out string examPeriod)
    {
        // Register exams for a student
        student = new Student { Index = "16947", FullName = "Danilo Terzic" };
        Subject[] subjects = new Subject[]
        {
                new Subject { Code = "MAT101", Name = "Mathematics" },
                new Subject { Code = "PHY201", Name = "Physics" }
        };
        examPeriod = "Fall 2023";
        OperationResult registrationResult = examRegistrationServiceClient.RegisterExam(student, subjects, examPeriod);
        if (registrationResult.IsError)
        {
            Console.WriteLine("Registration Result: " + registrationResult.ErrorMessage);
        }
        else
        {
            Console.WriteLine("Registration Result: " + registrationResult.SuccessMessage);
        }
    }

    private static void GetStudentExams(ExamRegistrationServiceClient examRegistrationServiceClient, Student student, string examPeriod)
    {
        // Get exams for a student in an exam period
        Exam[] studentExams = examRegistrationServiceClient.GetStudentExams(student.Index, examPeriod);
        Console.WriteLine("Student Exams:");
        foreach (var exam in studentExams)
        {
            Console.WriteLine($"Subject: {exam.Subject.Name}, Exam Period: {exam.ExamPeriod}");
        }
    }

    private static Subject GetStudentCountForExam(ExamRegistrationServiceClient examRegistrationServiceClient, string examPeriod)
    {
        // Get student count for an exam
        Subject examSubject = new Subject { Code = "MAT101", Name = "Mathematics" };
        int studentCount = examRegistrationServiceClient.GetStudentCountForExam(examPeriod, examSubject);
        Console.WriteLine($"Student Count for {examSubject.Name}: {studentCount}");
        return examSubject;
    }

    private static void GetStudentsForExam(ExamRegistrationServiceClient examRegistrationServiceClient, string examPeriod, Subject examSubject)
    {
        // Get students for an exam
        Student[] studentsForExam = examRegistrationServiceClient.GetStudentsForExam(examPeriod, examSubject);
        Console.WriteLine("Students for Mathematics Exam:");
        foreach (var studentForExam in studentsForExam)
        {
            Console.WriteLine($"Index: {studentForExam.Index}, Name: {studentForExam.FullName}");
        }
    }

    private static void UnregisterExam(ExamRegistrationServiceClient examRegistrationServiceClient, Student student)
    {
        // Unregister an exam for a student
        Subject subjectToUnregister = new Subject { Code = "MAT101", Name = "Mathematics" };
        OperationResult unregistrationResult = examRegistrationServiceClient.UnregisterExam(student, subjectToUnregister);
        if (unregistrationResult.IsError)
        {
            Console.WriteLine("Unregistration Result: " + unregistrationResult.ErrorMessage);
        }
        else
        {
            Console.WriteLine("Unregistration Result: " + unregistrationResult.SuccessMessage);
        }
    }
  }
}

namespace ClientConsoleApp
{
    internal class Program
    {
        static void Main(string[] args)
        {
            new ExamRegistrationConsoleClient();
        }
    }
}
```